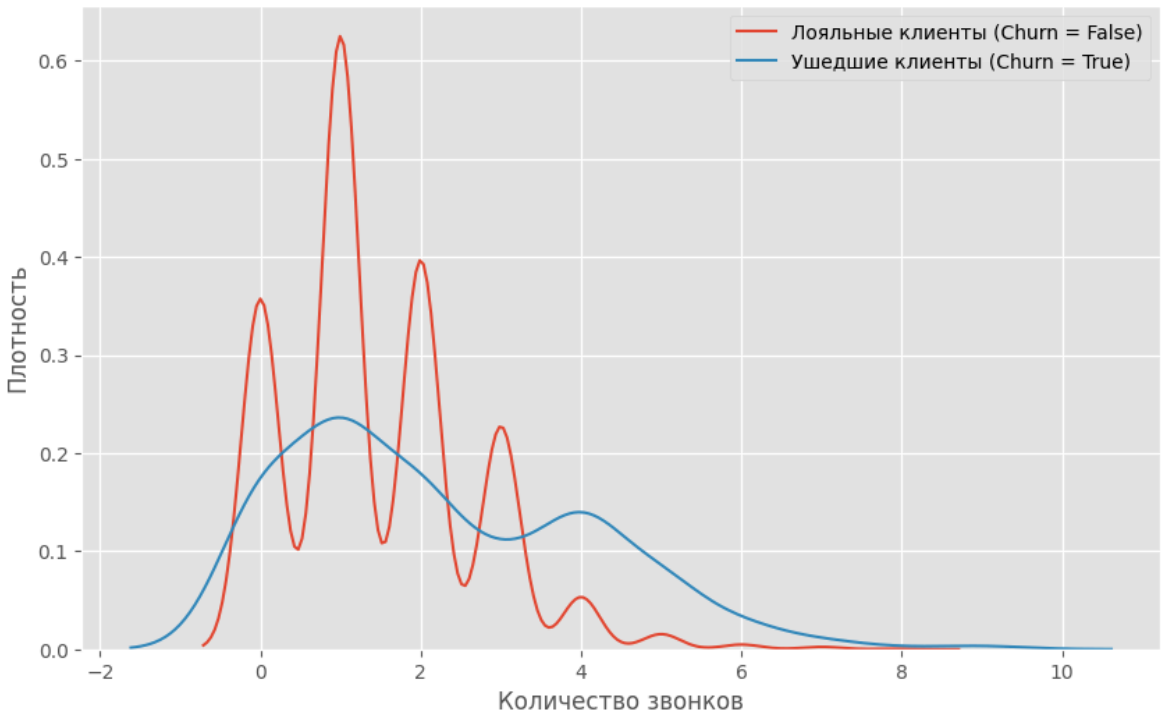
```
import pandas as pd  # pandas для работы с таблицами
from matplotlib import pyplot as plt  # графики
plt.style.use('ggplot')  # стиль графиков
plt.rcParams['figure.figsize'] = 10, 6  # размер окна графиков
import seaborn as sns  # для графиков
%matplotlib inline

URL = 'https://drive.google.com/uc?id=1U9LC5ECI2_Jvq4FrZNCOn3gCT7T5Y5DD'
telecom_data = pd.read_csv(URL)  # загружаем данные в таблицу

# выбираем и отображаем звонки (столбец 'Customer service calls') лояльных к
sns.kdeplot(telecom_data[telecom_data['Churn'] == False]['Customer service c
sns.kdeplot(telecom_data[telecom_data['Churn'] == True]['Customer service ca

plt.xlabel('Количество звонков')  # подпись оси X
plt.ylabel('Плотность')  # подпись оси Y
plt.legend()  # отображаем легенду
plt.show()  # отображаем график
```




```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
plt.style.use('ggplot')
plt.rcParams['figure.figsize'] = 10, 6
import seaborn as sns
%matplotlib inline

# Загружаем данные
URL = 'https://drive.google.com/uc?id=1U9LC5ECI2_Jvq4FrZNCOn3gCT7T5Y5DD'
telecom_data = pd.read_csv(URL)

# --- ФУНКЦИИ ---
```

```
def get_bootstrap_samples(data, n_samples, n_data):
    """Генерация бутстрэп-выборок"""
    indices = np.random.randint(0, len(data), (n_samples, n_data))
    samples = data[indices]
    return samples

def stat_intervals(stat, alpha):
    """Доверительный интервал для заданного уровня значимости"""
    boundaries = np.percentile(stat, [100 * alpha / 2., 100 * (1 - alpha / 2
    return boundaries

# --- ПАРАМЕТРЫ ---
n_samples = 1000   # число бутстрэп-выборок
n_data = 500       # размер каждой выборки
np.random.seed(0)  # для воспроизводимости

# --- ДАННЫЕ ---
loyal_calls = telecom_data[telecom_data['Churn'] == False]['Customer service
churn_calls = telecom_data[telecom_data['Churn'] == True]['Customer service

# --- БУТСТРЭП ---
loyal_mean_scores = [np.mean(sample) for sample in get_bootstrap_samples(loy
churn_mean_scores = [np.mean(sample) for sample in get_bootstrap_samples(chu

# --- РЕЗУЛЬТАТЫ ---
print("Service calls from loyal:  mean interval", stat_intervals(loyal_mean_
print("Service calls from churn:   mean interval", stat_intervals(churn_mean
print()
print("Service calls from loyal:  mean ", np.mean(loyal_calls))
print("Service calls from churn:   mean ", np.mean(churn_calls))
```

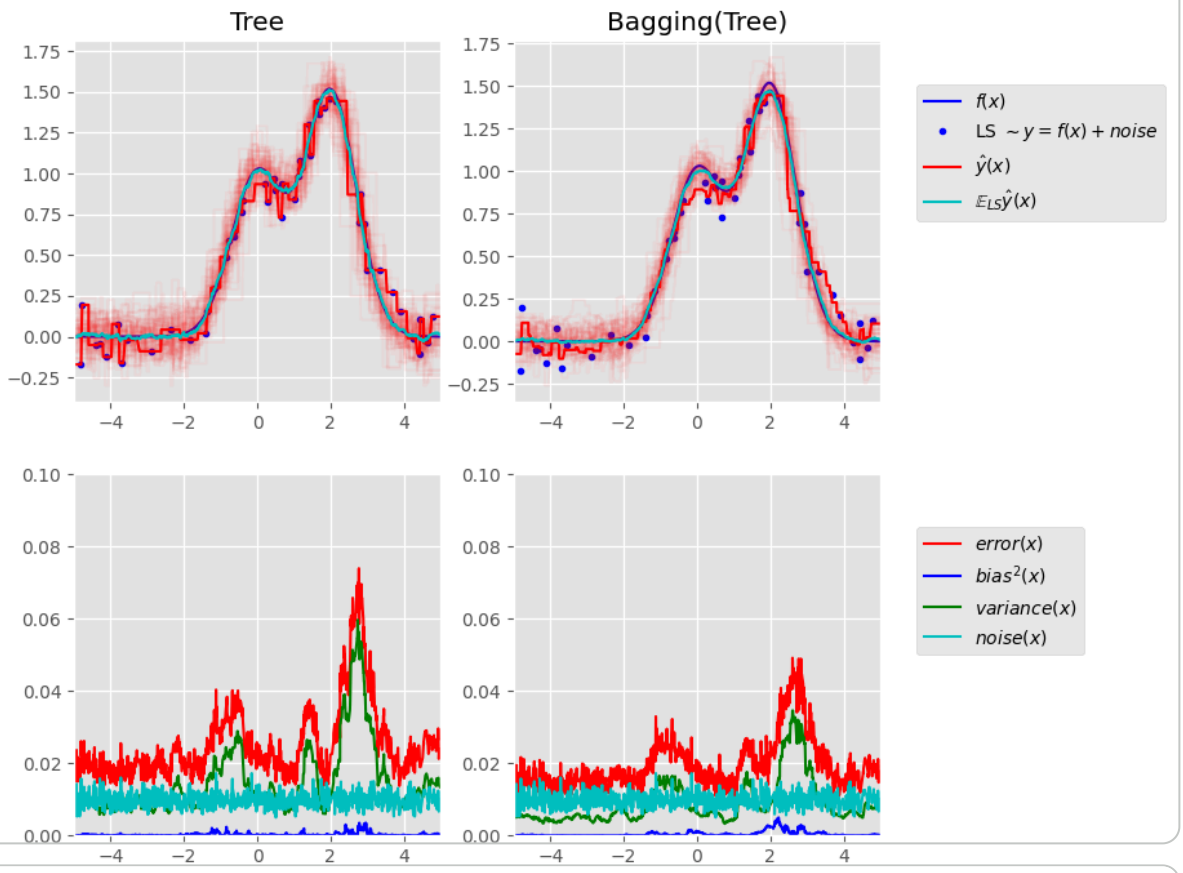
```
Service calls from loyal:  mean interval [1.35  1.556]
Service calls from churn:   mean interval [2.06   2.3961]

Service calls from loyal:  mean  1.4498245614035088
Service calls from churn:   mean  2.229813664596273
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor

# ------------------------------------------------
# Параметры
# ------------------------------------------------
n_repeat = 50          # число итераций для генерации шума
n_train  = 50          # число примеров в обучающей выборке
n_test   = 1000        # число примеров в тестовой выборке
noise    = 0.1         # стандартное отклонение шума
np.random.seed(0)      # для воспроизводимости

# ------------------------------------------------
# Модели
# ------------------------------------------------
estimators = [
```

```
    ("Tree", DecisionTreeRegressor()),
    ("Bagging(Tree)",
     BaggingRegressor(DecisionTreeRegressor(),
                      n_estimators=10,
                      max_samples=1.0))
]
n_estimators = len(estimators)

# ------------------------------------------------
# Истинная функция
# ------------------------------------------------
def f(x):
    x = x.ravel()
    return np.exp(-x ** 2) + 1.5 * np.exp(-(x - 2) ** 2)

# ------------------------------------------------
# Генерация данных
# ------------------------------------------------
def generate(n_samples, noise, n_repeat=1):
    X = np.random.rand(n_samples) * 10 - 5      # от -5 до 5
    X = np.sort(X)
    if n_repeat == 1:
        y = f(X) + np.random.normal(0.0, noise, n_samples)
    else:
        y = np.zeros((n_samples, n_repeat))
        for i in range(n_repeat):
            y[:, i] = f(X) + np.random.normal(0.0, noise, n_samples)
    X = X.reshape((n_samples, 1))
    return X, y

# ------------------------------------------------
# Обучающие наборы (по n_repeat раз)
# ------------------------------------------------
X_train = []
y_train = []
for i in range(n_repeat):
    X, y = generate(n_samples=n_train, noise=noise)
    X_train.append(X)
    y_train.append(y)

# Тестовый набор (один, но с n_repeat зашумлёнными функциями)
X_test, y_test = generate(n_samples=n_test, noise=noise, n_repeat=n_repe

# ------------------------------------------------
# Графики
# ------------------------------------------------
plt.figure(figsize=(10, 8))

for n, (name, estimator) in enumerate(estimators):
    # предсказания модели на всех повторениях
    y_predict = np.zeros((n_test, n_repeat))

    for i in range(n_repeat):
        estimator.fit(X_train[i], y_train[i])
        y_predict[:, i] = estimator.predict(X_test)
```

```
        # ---- Bias² + Variance + Noise ----
        y_error = np.zeros(n_test)
        for i in range(n_repeat):
            for j in range(n_repeat):
                y_error += (y_test[:, j] - y_predict[:, i]) ** 2
        y_error /= (n_repeat * n_repeat)

        y_noise = np.var(y_test, axis=1)
        y_bias  = (f(X_test) - np.mean(y_predict, axis=1)) ** 2
        y_var   = np.var(y_predict, axis=1)

        print("{0:15}: {1:.4f} (error) = {2:.4f} (bias^2) "
              "+ {3:.4f} (var) + {4:.4f} (noise)".format(
                  name,
                  np.mean(y_error),
                  np.mean(y_bias),
                  np.mean(y_var),
                  np.mean(y_noise)))

        # ---- График предсказаний ----
        plt.subplot(2, n_estimators, n + 1)
        plt.plot(X_test, f(X_test), "b", label=r"$f(x)$")
        plt.plot(X_train[0], y_train[0], ".b", label=r"LS $\sim y = f(x)+no
        for i in range(n_repeat):
            if i == 0:
                plt.plot(X_test, y_predict[:, i], "r", label=r"$\hat{y}(x)$
            else:
                plt.plot(X_test, y_predict[:, i], "r", alpha=0.05)

        plt.plot(X_test, np.mean(y_predict, axis=1), "c",
                 label=r"$\mathbb{E}_{LS} \hat{y}(x)$")
        plt.xlim([-5, 5])
        plt.title(name)

        if n == n_estimators - 1:
            plt.legend(loc=(1.1, .5))

        # ---- График разложения ошибки ----
        plt.subplot(2, n_estimators, n_estimators + n + 1)
        plt.plot(X_test, y_error, "r", label=r"$error(x)$")
        plt.plot(X_test, y_bias,  "b", label=r"$bias^2(x)$")
        plt.plot(X_test, y_var,   "g", label=r"$variance(x)$")
        plt.plot(X_test, y_noise, "c", label=r"$noise(x)$")
        plt.xlim([-5, 5])
        plt.ylim([0, 0.1])

        if n == n_estimators - 1:
            plt.legend(loc=(1.1, .5))

    plt.subplots_adjust(right=0.75)
    plt.show()
```

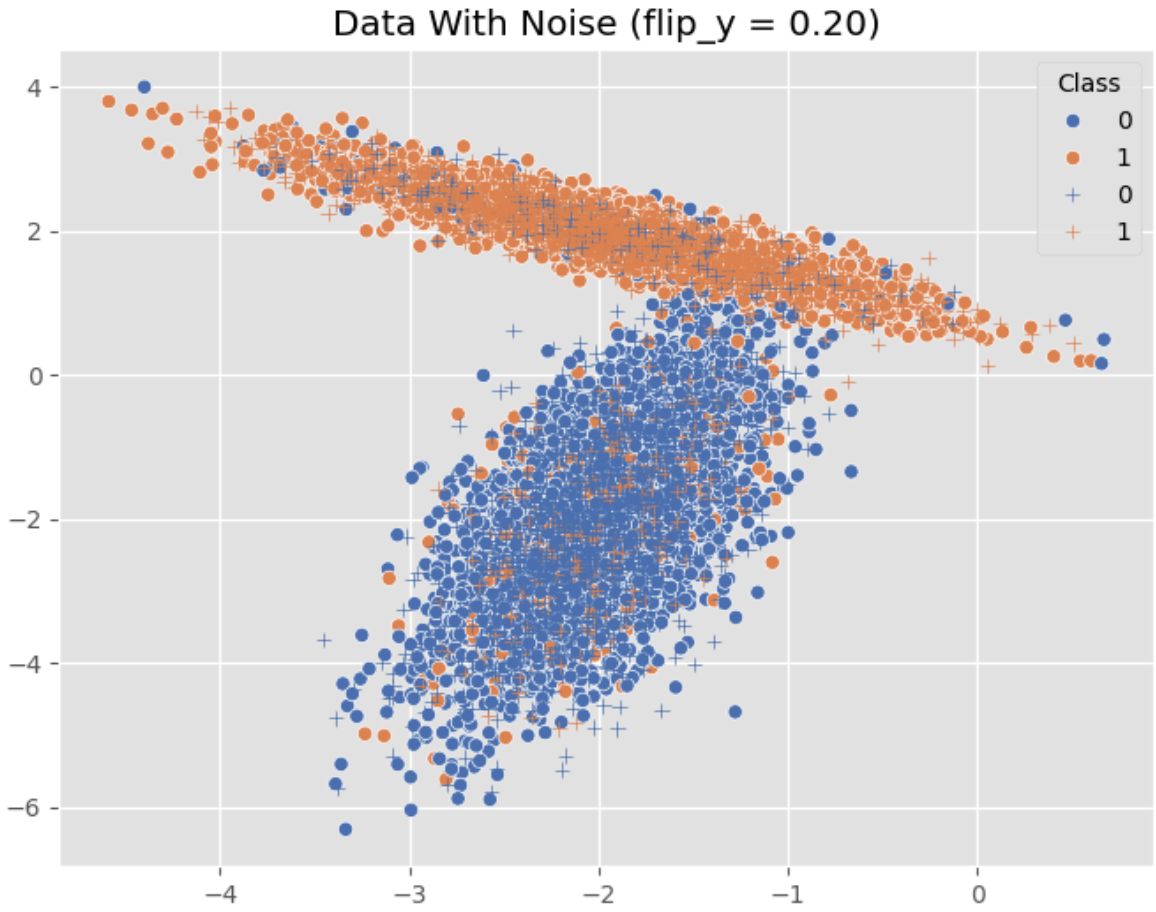
```
Tree           : 0.0255 (error) = 0.0003 (bias^2) + 0.0152 (var) + 0.0098 (no
Bagging(Tree)  : 0.0196 (error) = 0.0004 (bias^2) + 0.0092 (var) + 0.0098 (no
```



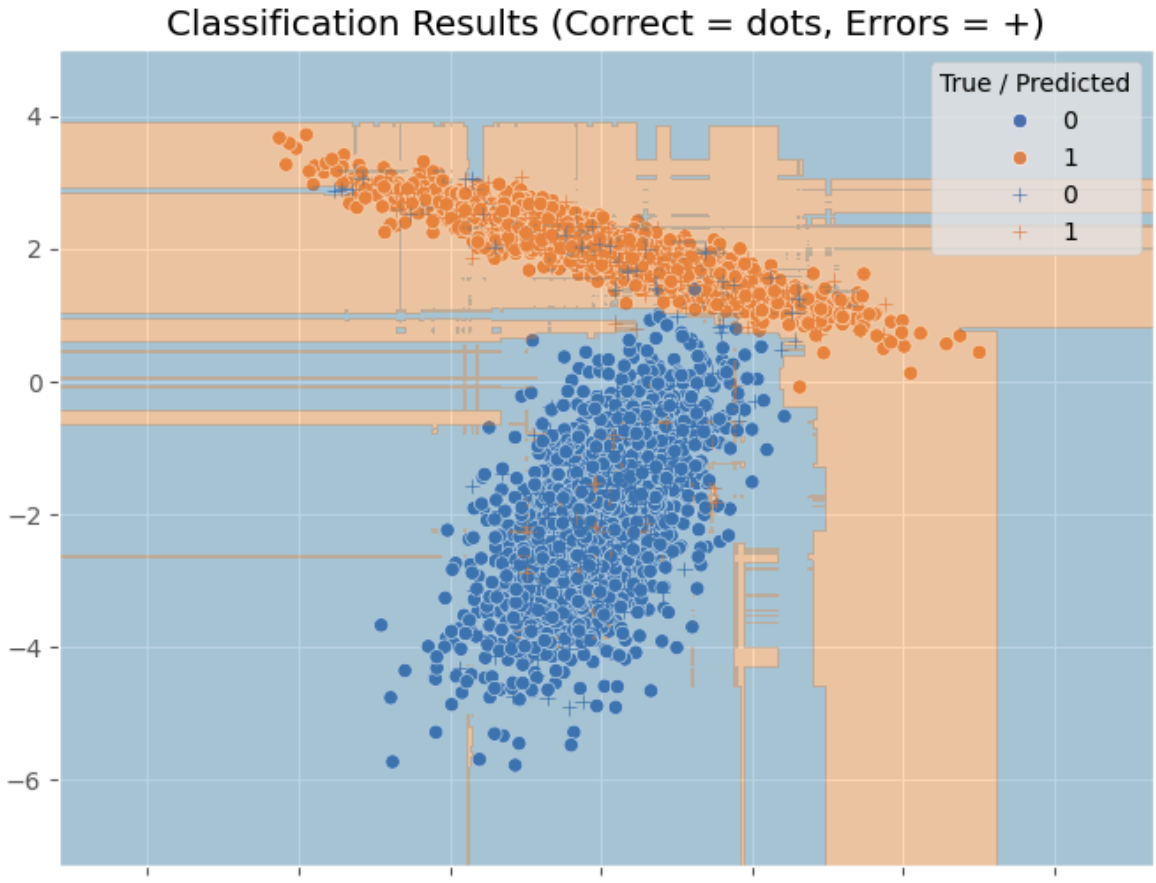

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

# ----------------------------------------------------
# Функция: визуализация разделяющей поверхности
# ----------------------------------------------------
def Div_plate(clf, flip_level):
    # Генерация данных
    X, y = make_classification(
        n_samples=10000,
        n_features=2,
        n_informative=2,
        n_redundant=0,
        n_repeated=0,
        n_classes=2,
        n_clusters_per_class=1,
        class_sep=2,
        flip_y=flip_level,        # уровень шума
        weights=[0.5, 0.5],
        random_state=17
    )

    # Разделение на train/test
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.33, random_state=42
```

```
            X, y, test_size=0.33, random_state=42
        )

        # --- График 1: Исходные данные ---
        plt.figure(figsize=(8, 6))
        sns.scatterplot(x=X_train[:, 0], y=X_train[:, 1], hue=y_train, pale
        sns.scatterplot(x=X_test[:, 0],  y=X_test[:, 1],  hue=y_test,  pale
        plt.title("Data With Noise (flip_y = {:.2f})".format(flip_level))
        plt.legend(title='Class', loc='upper right')
        plt.show()

        # --- Обучение классификатора ---
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        score = clf.score(X_test, y_test)
        print(f'Score = {score:.4f}')

        # --- График 2: Правильные/неправильные предсказания ---
        ind_correct = (y_test == y_pred)   # правильно классифицированные
        ind_wrong   = (y_test != y_pred)   # ошибки

        plt.figure(figsize=(8, 6))
        ax = plt.gca()

        # Правильно классифицированные
        sns.scatterplot(x=X_test[ind_correct, 0], y=X_test[ind_correct, 1],
                        hue=y_test[ind_correct], palette='deep', ax=ax, lege

        # Ошибки (с предсказанным классом)
        sns.scatterplot(x=X_test[ind_wrong, 0], y=X_test[ind_wrong, 1],
                        hue=y_pred[ind_wrong], palette='deep', marker='+', a

        plt.title("Classification Results (Correct = dots, Errors = +)")
        plt.legend(title='True / Predicted', loc='upper right')

        # --- Разделяющая поверхность ---
        plot_step = 0.01
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

        xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step),
                             np.arange(y_min, y_max, plot_step))

        Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
        Z = Z.reshape(xx.shape)

        plt.contourf(xx, yy, Z, levels=1, colors=['#1f77b4', '#ff7f0e'], alp
        plt.show()

    # ------------------------------------------------
    # Запуск примера
    # ------------------------------------------------
    clf = BaggingClassifier(
        DecisionTreeClassifier(),
        n_estimators=10,
        max_samples=0.5,
```

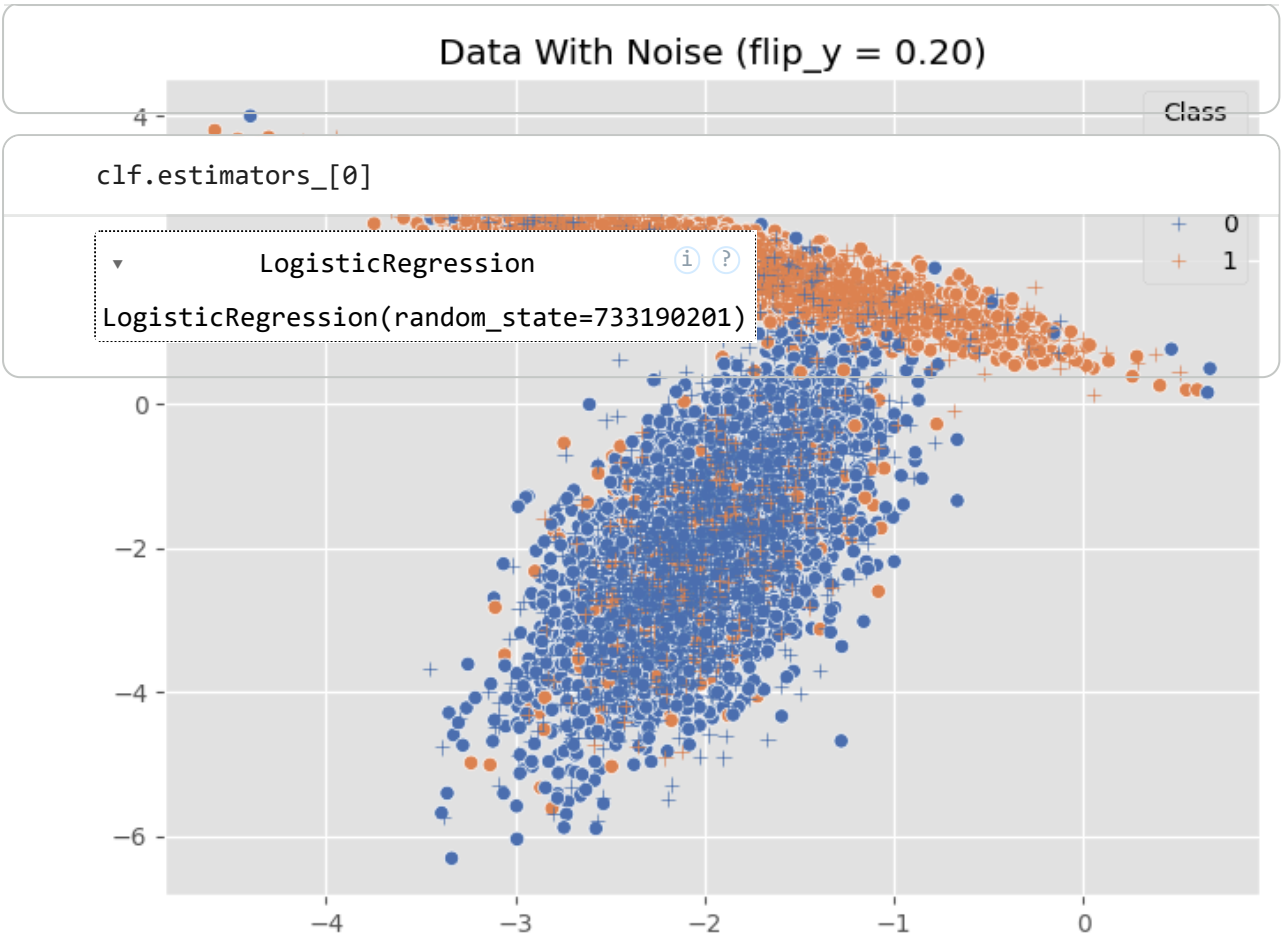
```
    random_state=42
)

flip_level = 0.2
Div_plate(clf, flip_level)
```




```
Score = 0.8830
```



```
from sklearn.ensemble import BaggingClassifier #
from sklearn.linear_model import LogisticRegression # модель для регрессии
#from sklearn.linear_model import LinearRegression # модель для регрессии
clf=BaggingClassifier(LogisticRegression(), n_estimators=10,max_samples=0.5,
#clf=BaggingClassifier(LinearRegression(), n_estimators=10,max_samples=0.5,
flip_level=0.2
Div_plate(clf,flip_level)
```




```
clf.estimators_[0]
```

```
LogisticRegression(random_state=733190201)
```

```
Score = 0.8827
```